

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Smart Energy Grid Analytics Using MQTT, TimescaleDB, Grafana and Python

Advanced Big Data Analytics

Student Name	Student ID
Rubagumya Brave	101392
Rwagasana Feliciene	101364
Umuhoza Diane	101286

Program: MSc in Big Data Analytics
Course: Advanced Big Data Analytics
Lecturer: Temitope Oguntade
GitHub: https://github.com/Rubagumyabrave/Big_data_Analysis_Assignment_I_Smart_grid
Date: October 15, 2026

Contents

1 Introduction **4**

1.1 Project Repository 4

2 Business Problem **4**

3 Dataset Description **5**

3.1 Data Source 5

3.2 Data Dictionary 6

4 Environment **6**

5 Ingestion Pipeline **8**

5.1 MQTT Pilot Design 8

5.2 Pilot Results 8

5.3 Full Historical Load 10

6 Data Quality and Cleaning **11**

6.1 Quality Measurement 11

6.2 Cleaning Duplicates 12

7 Storage Design: Chunk Intervals **13**

7.1 Configurations 13

7.2 Benchmark Results 13

8 Compression **14**

8.1 Configurations 14

8.2 Storage Results 15

8.3 Query Performance 15

9 Analytical Results **16**

9.1 Demand Patterns 17

9.2 Regions and Customer Types 17

9.3 Reporting Quality 18

9.4 Anomaly Detection 19

10 Continuous Aggregates **20**

10.1 Design 20
10.2 Freshness and Delayed Arrivals 20

11 Dashboards **21**
11.1 Raw and Aggregate Dashboards 21
11.2 Verification 22
11.3 Performance 22

12 Day-Ahead Forecasting **24**
12.1 Dataset and Features 24
12.2 Baseline and Model 24
12.3 Test Results 25

13 Business Insights **25**

14 Reproducibility and Contributions **27**
14.1 Reproducibility 27
14.2 Group Contributions 28

15 Limitations **28**

16 Conclusion **28**

17 References **29**

Execution Evidence **29**

1 Introduction

A utility operates 4,000 smart meters spread over ten regions. Every meter reports one reading every 15 minutes, and, as in real networks, some readings arrive late, some arrive twice and some never arrive. Over 28 days (1–28 September) this produces about 10.7 million readings. This report presents a Big Data project that builds the complete data path for that grid on a single machine and uses it to answer operational questions.

Readings are published through an **EMQX MQTT broker**, stored in **PostgreSQL with the TimescaleDB extension**, analysed with **Python and SQL**, pre-aggregated with **continuous aggregates**, displayed on **Grafana** dashboards, and used to train a **day-ahead forecasting model** with scikit-learn. All data comes from the supplied, unchanged generator `smart_meter_simulator.py`, and all times are Central Africa Time (CAT, Africa/Kigali).

1.1 Project Repository

All source code, SQL scripts, results and execution evidence are available at:

https://github.com/Rubagumyabrave/Big_data_Analysis_Assignment_I_Smart_grid

Path	Contents
`sql/`	13 SQL scripts: database, schema, checks, quality, cleaning, chunks, compression, aggregates
`src/`	Python scripts: MQTT publisher/subscriber, loader, benchmarks, analysis, dashboards, forecast
`grafana/`	Provisioned data source and the two dashboard definitions
`results/`	Result summaries (`.md`), benchmark CSV files and logs
`results/figures/`	Charts produced by the analysis and forecast scripts
`Screenshoots/`	Execution evidence (63 screenshots, listed at the end of this report)
`.env.example`	Connection settings with placeholder passwords
`requirements.txt`	Python packages
`README.md`	Set-up and step-by-step reproduction instructions

Table 1: Repository structure

The full dataset is not committed (10.7 million rows, about 1.2 GB as CSV). It is regenerated with `python src/ingest.py`, because the generator is deterministic. The `.env` file holding real passwords is excluded by `.gitignore`.

2 Business Problem

A grid operator needs reliable data before it can make decisions. This project addresses seven questions:

- **Reliable ingestion:** Can every reading travel from the meter, through the MQTT broker, into the database without being lost?
- **Data quality:** How many readings are duplicated, delayed or missing, and how should they be handled?
- **Storage design:** Which chunk interval and which compression layout give the best balance of storage and query speed?
- **Demand understanding:** When does demand peak, how do weekdays and weekends differ, and which regions and customer types drive consumption?
- **Anomaly detection:** Can abnormal spikes and drops in a meter's consumption be detected automatically?
- **Fast reporting:** How much faster are dashboards built on pre-computed aggregates, and do they give exactly the same answers?
- **Planning:** Can a regression model forecast tomorrow's hourly regional demand better than the rule "same as yesterday"?

3 Dataset Description

3.1 Data Source

The supplied generator provides three functions: `meter_metadata()` returns the 4,000 meters, `iter_readings()` streams the readings in arrival order, and `dataset_info()` / `anomaly_events()` return reference counts and the 40 injected anomalies used to check our results.

Item	Value
Meters	4,000 (400 per region, 10 regions)
Period	1 Sep 00:00 – 28 Sep 23:45 CAT (28 days)
Reading interval	15 minutes (2,688 slots per meter)
Planned readings	10,752,000
Emitted readings (including duplicates)	10,696,848
Customer types	residential 2,584; commercial 1,010; industrial 406

Table 2: Dataset overview

3.2 Data Dictionary

Column	Type	Meaning	Unit
energy_live and energy_readings (same eight columns)			
`meter_id`	BIGINT	Meter identifier (1000000000–1000003999)	–
`event_time`	TIMESTAMPTZ	Start of the 15-minute interval	CAT
`received_time`	TIMESTAMPTZ	Simulated arrival time	CAT
`power_kw`	DOUBLE	Average power over the interval	kW
`voltage_v`	DOUBLE	Supply voltage	V
`current_a`	DOUBLE	Current	A
`frequency_hz`	DOUBLE	Grid frequency	Hz
`energy_kwh`	DOUBLE	Energy in the interval ($\approx$ power $\times$ 0.25 h)	kWh
meters (4,000 rows)			
`meter_id`	BIGINT, PK	Meter identifier	–
`region`	TEXT	Region 1 to Region 10	–
`customer_type`	TEXT	residential, commercial or industrial	–
`base_power_kw`	DOUBLE	Typical base load	kW
`power_factor`	DOUBLE	Real / apparent power (0.95)	ratio

Table 3: Data dictionary

Three tables were created by `sql/02_schema.sql`: `energy_live` (a regular table for the live MQTT pilot), `energy_readings` (a hypertable with one-day chunks for the full history) and `meters`. The reading tables have no primary key, so duplicates can be stored first and investigated later. `src/load_meters.py` loaded the 4,000 meter records (`SELECT count(*) FROM meters` = 4,000).

4 Environment

EMQX and PostgreSQL/TimescaleDB run as Docker containers on one laptop. The database `smart_grid` has the `timescaledb` extension enabled and its time zone set to `Africa/Kigali`, so daily buckets start at CAT midnight.

Component	Specification
CPU	Intel Core i7-6600U @ 2.60 GHz, 2 cores / 4 logical processors
RAM (host)	15,784 MB
Operating system	Windows 10 Pro, version 10.0.19045
Storage	SSD (C:, Docker data and database files); HDD (D:, project folder and CSV)
Docker VM	WSL2, 4 CPUs, 7.47 GiB RAM, no per-container limits
Python	3.11.9 (virtual environment)
MQTT broker	EMQX 6.3.1
Database	PostgreSQL 17.11 + TimescaleDB 2.30.2
shared_buffers / work_mem	1911 MB / 15294 kB
Parallel workers per gather / jit	2 / off
Dashboards	Grafana OSS (Docker, provisioned from files)

Table 4: Environment configuration



Figure 1: CPU, EMQX version, Docker VM resources, container limits and PostgreSQL settings

The database shares a 4-CPU, 7.5 GB virtual machine with the broker. This matters for the benchmarks: a parallel query can use at most three processes (leader + 2 workers), and tables larger than 1.9 GB cannot stay in PostgreSQL's own cache.

5 Ingestion Pipeline

5.1 MQTT Pilot Design

Before loading millions of rows, a pilot proved that readings survive the MQTT path. `src/publisher.py` takes the first 2,000 readings with `itertools.islice(iter_readings(), 2000)` and publishes each as JSON to the topic `energy/meters/{meter_id}` with QoS 1 and no retained messages, counting the broker acknowledgements (PUBACK). `src/subscriber.py`, started first, subscribes to `energy/meters/#` at QoS 1 and inserts every message into `energy_live`.

5.2 Pilot Results

Stage	**Run 1**	**Run 2 (fixed)**
Readings taken from the generator	2,000	2,000
Messages acknowledged by the broker	2,000	2,000
Messages received by the subscriber	1,817	2,000
Rows in `energy_live`	1,817	2,000

Table 5: Pilot counts at every stage

In run 1, 183 messages were lost *inside the broker*: the publisher had 2,000 acknowledgements, the subscriber logged no errors, and EMQX reported `delivery.dropped.queue_full = 183`. The publisher sent about 1,200 messages per second while the subscriber wrote to the database inside its MQTT callback, so it stopped reading while it waited; its broker queue filled and EMQX discarded the overflow. A PUBACK only proves that the broker received a message, not that a subscriber got it.

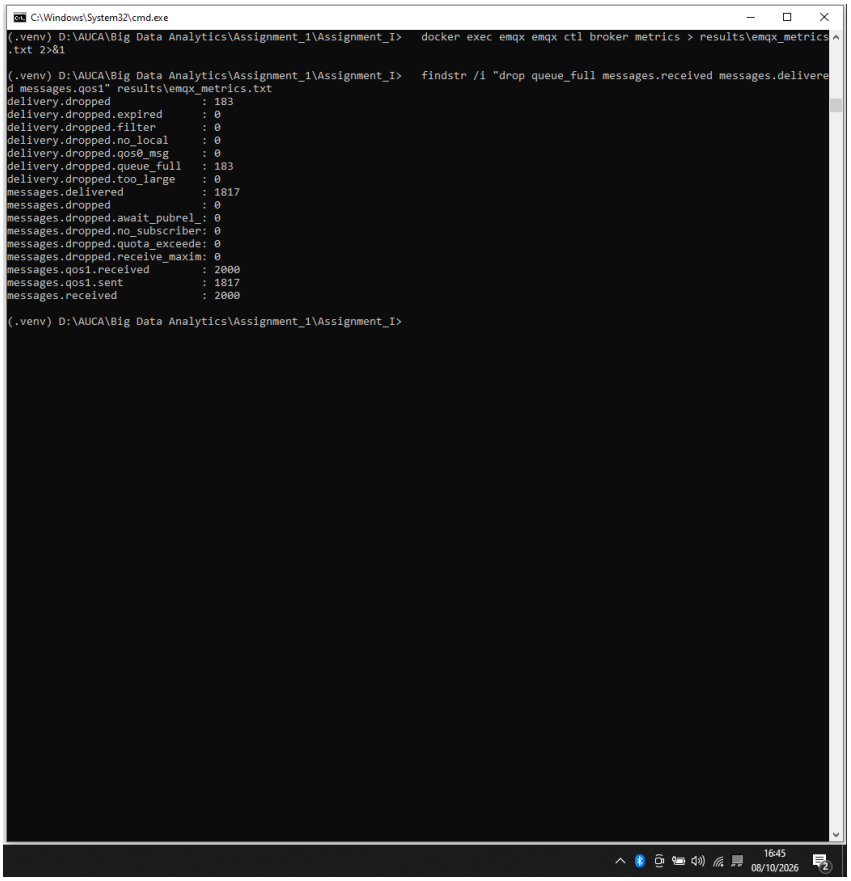


Figure 2: Run 1: EMQX metrics showing 183 messages dropped because the subscriber queue was full

Fix. The subscriber now reads messages in one thread and writes to the database in a second thread through a queue, so reading never waits for the database (largest backlog: 133 rows), and the publisher is paced at 300 messages per second. Run 2 reached **2,000 at every stage** with no new drops. The 2,000 rows include 6 deliberate generator duplicates.

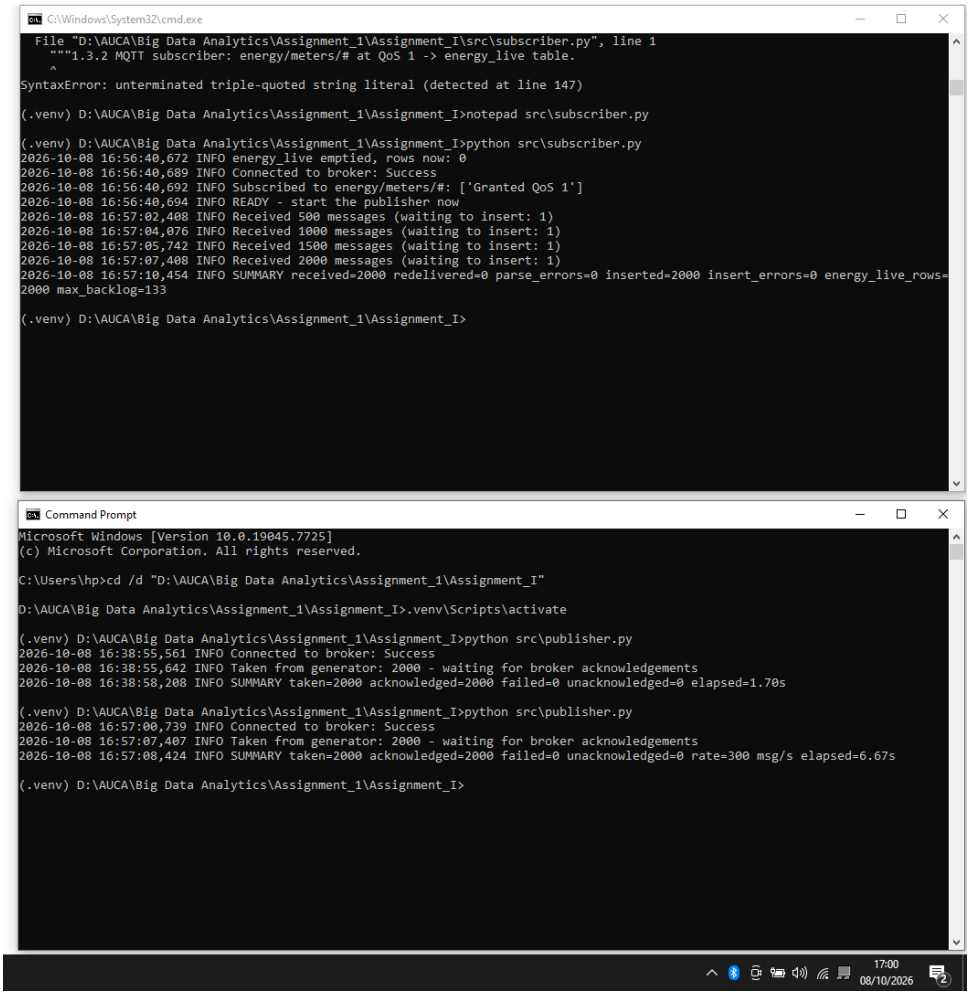


Figure 3: Run 2: publisher (2,000 taken, 2,000 acknowledged) and subscriber (2,000 received)

5.3 Full Historical Load

Publishing 10.7 million messages through MQTT would take hours, so `src/ingest.py` writes the complete `iter_readings()` stream to a CSV file in batches and bulk-loads it with PostgreSQL `COPY` in one transaction, duplicates included.

Count / timing	Value
Expected readings (`dataset_info()`)	10,696,848
Readings consumed / CSV rows / table rows	10,696,848 / 10,696,848 / 10,696,848
CSV generation	193.4 s
Database load (first write to commit)	166.2 s
Total	368.8 s
Chunks / total size	29 / 1,042.7 MB

Table 6: Full load: counts and timings

All four counts are identical, so nothing was lost. There are 29 chunks rather than 28 because TimescaleDB aligns daily chunks to UTC midnight (02:00 CAT).

6 Data Quality and Cleaning

6.1 Quality Measurement

`sql/04_data_quality.sql` builds the planned grid (4,000 meters × 2,688 slots) and compares it with the observed (meter_id, event_time) pairs. A reading is *delayed* when it arrived more than 15 minutes after its interval started.

Measure	Measured	Reference	Rate
Planned readings	10,752,000	10,752,000	–
Emitted readings	10,696,848	10,696,848	–
Unique readings	10,643,735	10,643,735	98.99 % complete
Duplicate copies	53,113	53,113	0.50 %
Delayed readings	213,090	213,090	2.00 %
Missing readings	108,265	108,265	1.01 %

Table 7: Data-quality counts against the generator's reference values

All six counts match the reference exactly. Even the worst meter (1000003622, Region 3, commercial) misses only 47 of 2,688 readings (1.7 %), and the worst meters are spread over regions and customer types, so the gaps are random rather than concentrated.

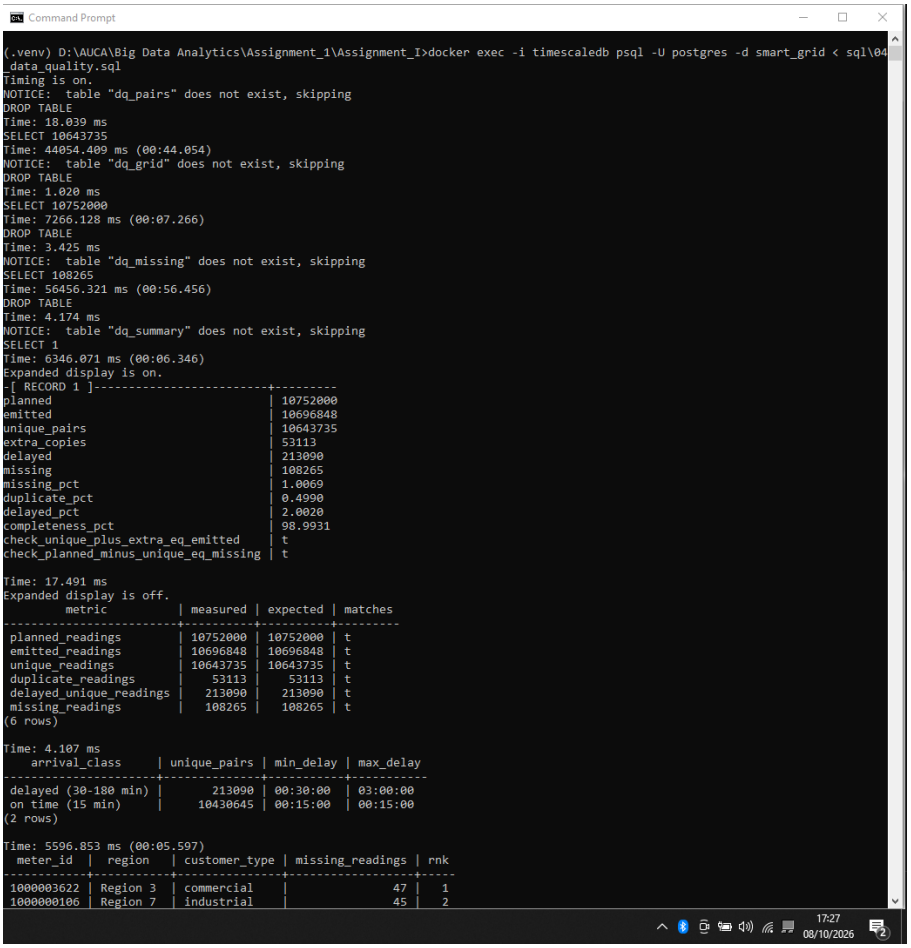


Figure 4: Data-quality counts and comparison with the reference values

6.2 Cleaning Duplicates

All 53,113 duplicate groups had exactly two copies, identical in every column, so there were no conflicting values to resolve. `sql/05_clean_duplicates.sql` numbers the copies with `row_number()` and deletes copies 2+ inside one transaction; a second run deletes nothing.

	Rows	Total energy (kWh)
Before cleaning	10,696,848	3,872,863.89
After cleaning	10,643,735	3,853,704.35
Removed	53,113	19,159.55

Table 8: Effect of duplicate cleaning

The final count equals the reference number of unique readings. Keeping duplicates would have overstated consumption by about 0.5 %.

Why missing readings are not stored as zero. A missing reading means the consumption is *unknown*, not zero. Zeros would understate totals, create false "sudden drop" anomalies, teach the forecast model fake low values, and hide the quality problem. The 108,265 gaps are

therefore left as gaps.

7 Storage Design: Chunk Intervals

7.1 Configurations

A hypertable is split into time "chunks". Small chunks let a query skip more data but add planning overhead; large chunks are cheaper to manage but force searches inside larger tables. The cleaned data was copied into three hypertables with identical indexes `(event_time DESC)` and `(meter_id, event_time DESC)`.

Table	Chunk interval	Chunks	Rows
`energy_readings_3h`	3 hours	225	10,643,735
`energy_readings`	1 day	29	10,643,735
`energy_readings_week`	7 days	5	10,643,735

Table 9: The three chunk configurations (identical rows and sums)

7.2 Benchmark Results

`src/benchmark.py` ran five typical queries on each table: one warm-up run, then five measured runs, reporting the median. No other jobs ran during the session, and every query returned identical results on all three tables.

Query (median, ms)	3 hours	1 day	7 days	Main reason
Q1 hourly average, one day	**117.8**	122.2	142.9	small chunks read whole
Q2 top 10 intervals, one week	285.9	**278.4**	325.2	daily chunks fit the week
Q3 energy per meter, all data	2,393.1	1,275.6	**1,270.9**	225 chunks: no parallel workers
Q4 one meter, one week	6.9	6.4	**2.8**	fewer index lookups
Q5 full scan	**902.4**	940.4	929.1	every row read anyway

Table 10: Warm-cache medians for the three chunk intervals

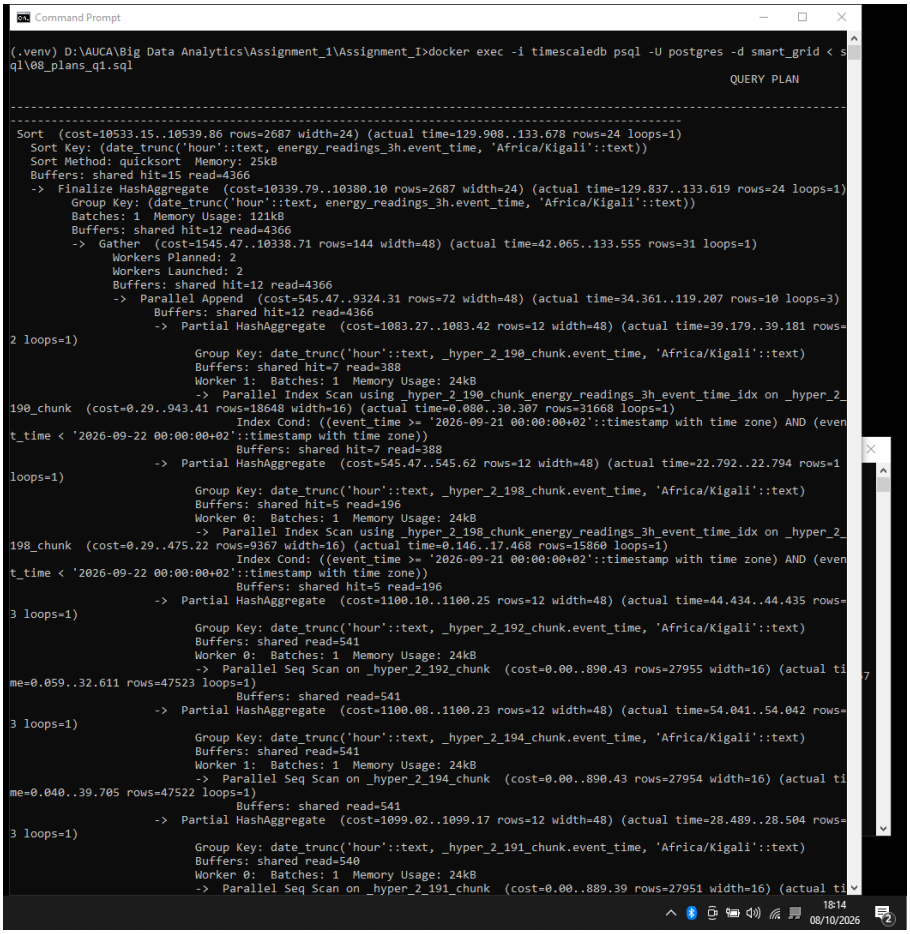


Figure 5: Benchmark medians for the three chunk intervals

Decision. One-day chunks are never the slowest and are within 2 % of the best on four of five queries. Three-hour chunks fail on whole-table grouping (Q3 is 1.88× slower) and 7-day chunks are weakest on short time windows. One-day chunks are used for the rest of the project.

8 Compression

8.1 Configurations

TimescaleDB can convert old chunks into a compressed, column-oriented format. Two layouts were compared on copies of the one-day table: **time-ordered** (`orderby event_time DESC`) and **meter-segmented** (`segmentby meter_id`, `orderby event_time DESC`). Each received a 48-hour compression policy, and all 29 chunks were compressed. Row counts and sums stayed identical.

8.2 Storage Results

Table	Total size	Ratio	Saved	Compression time
Uncompressed baseline	1,356.7 MB	1.00	–	–
Time-ordered	438.7 MB	3.09×	67.7 %	58.3 s
Meter-segmented	501.6 MB	2.70×	63.0 %	116.7 s

Table 11: Storage and compression cost

Time-ordered compression is smaller because it packs full 1,000-row batches, while meter segmentation makes batches of about 95 rows (one meter, one day) with more metadata.

8.3 Query Performance

Query (median, ms)	Baseline	Time-ordered	Meter-segmented
Q1 hourly average, one day	125.3	**90.6**	106.0
Q2 top 10 intervals, one week	263.7	186.2	**119.0**
Q3 energy per meter	1,304.6	525.3	**268.4**
Q4 one meter, one week	**3.4**	95.3	4.1
Q5 full scan	856.0	508.3	**425.1**

Table 12: Query medians on uncompressed and compressed tables

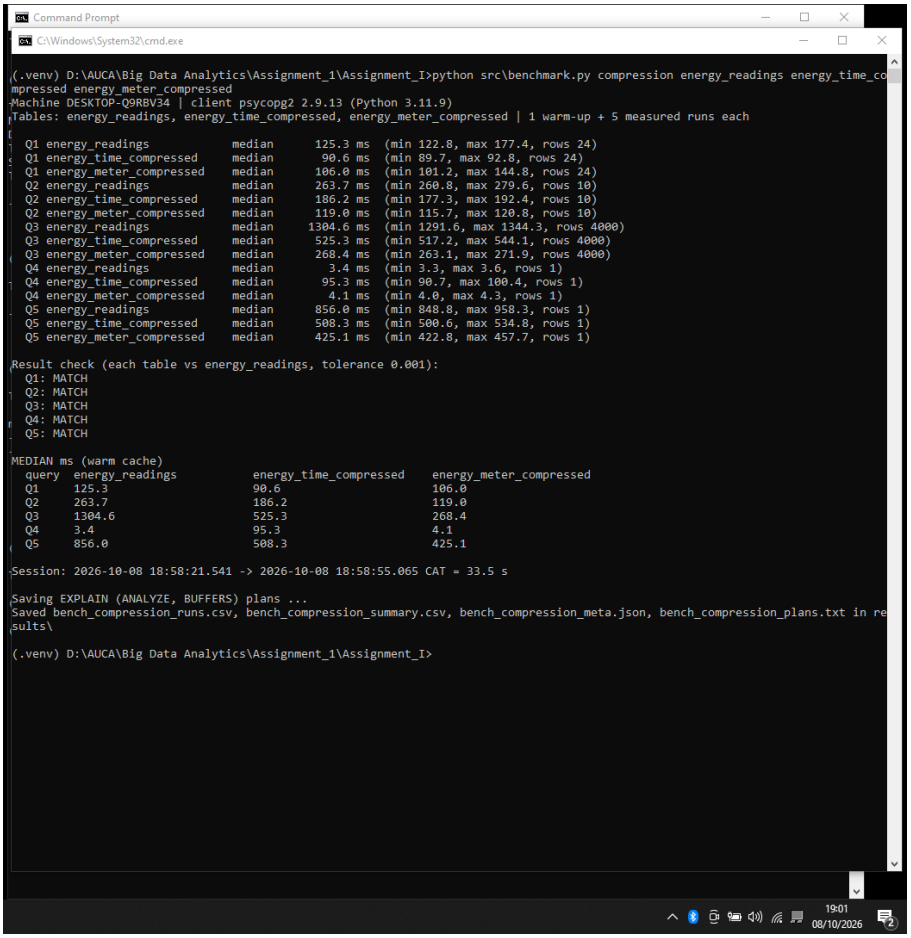


Figure 6: Benchmark medians: baseline vs the two compressed tables

Scans and aggregations became 15–79 % faster on both compressed tables because column storage reads only the needed columns. The single-meter lookup (Q4) decides the choice: time-ordered batches mix all meters, so finding one meter means decompressing every batch (95.3 ms, 28× slower), while meter segmentation filters on `meter_id` before decompressing (4.1 ms). **Recommendation:** compress chunks older than 48 hours, segmented by `meter_id`.

9 Analytical Results

All analysis uses the cleaned `energy_readings` table (10,643,735 readings).

9.1 Demand Patterns

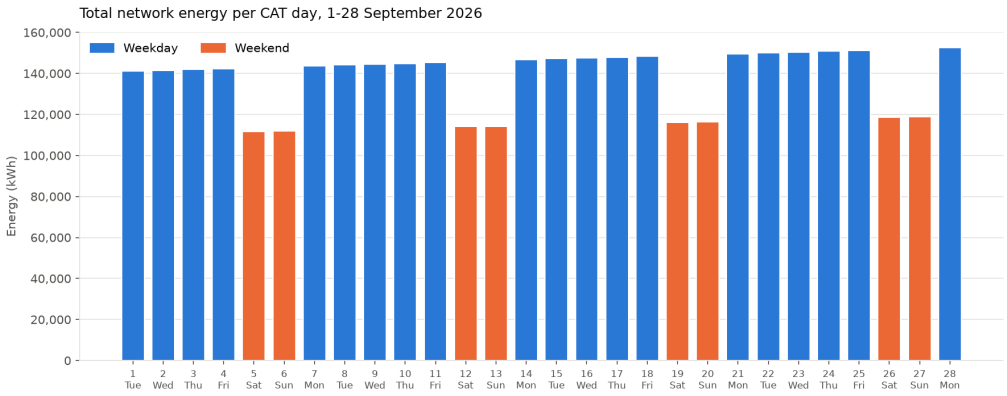


Figure 7: Daily network energy (weekend days highlighted)

Seven-day period	1–7 Sep	8–14 Sep	15–21 Sep	22–28 Sep
Energy (kWh)	934,199.1	953,715.0	973,230.0	992,560.3

Table 13: Network energy per seven-day period

An average weekday uses **146,571.5 kWh** and an average weekend day **115,284.2 kWh**, which is **21.3 % less**. Weekly energy grew steadily, **+6.2 %** from the first to the fourth week (about 2 % per week). The three highest 15-minute network demand intervals were all on Monday 28 September around midday (peak 7,993.7 kW at 13:00, 1.39× the average).

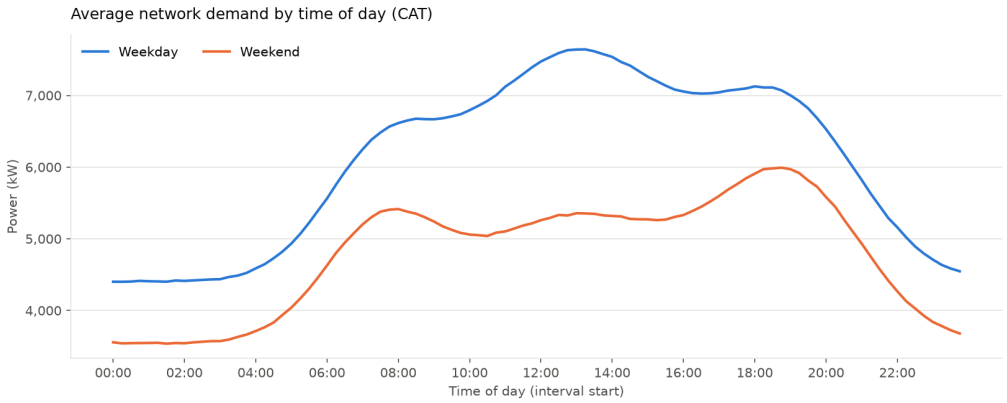


Figure 8: Average 15-minute network demand profile, weekday vs weekend

Weekdays peak around 13:00, while weekends peak later and lower (around 18:45), which points to commercial and industrial load during working hours.

9.2 Regions and Customer Types

Energy was divided by the number of *registered* meters in each group so that groups of different sizes can be compared fairly.

Customer type	Meters	% of meters	% of energy	kWh per meter
Industrial	406	10.15	57.62	5,469.6
Commercial	1,010	25.25	21.42	817.3
Residential	2,584	64.60	20.96	312.5

Table 14: Energy by customer type

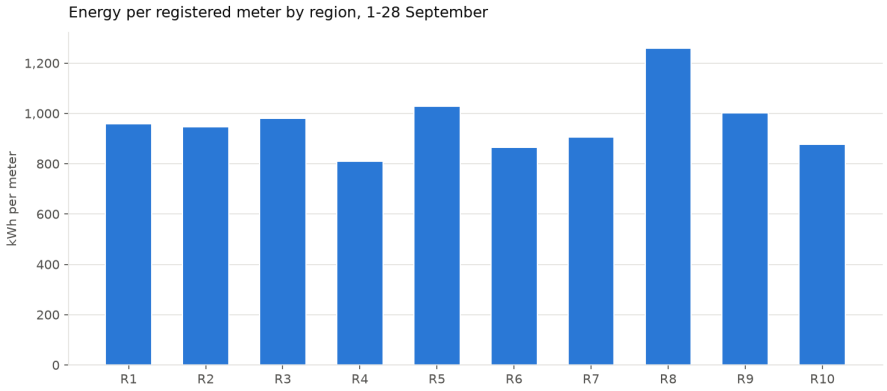


Figure 9: Energy per registered meter by region

An industrial meter uses **17.5**× the energy of a residential one. Every region has 400 meters, yet energy per meter varies by up to 1.55× (Region 8: 1,258.7 kWh vs Region 4: 811.0 kWh). This is explained by the customer mix: the correlation with each region's industrial share is **r = 0.97**.

9.3 Reporting Quality

Measure	Value
Network completeness	98.99 %
Range across regions	98.97 % (Region 6) – 99.01 % (Region 8)
21 Sep total known at midnight	149,400.545 kWh
21 Sep final total (all arrivals)	149,552.723 kWh
Difference caused by late arrivals	152.178 kWh (0.10 %)

Table 15: Completeness and on-time vs final daily totals

Missing readings are spread evenly across regions, so regional comparisons are not biased. **Delays** make an early report temporarily low (the last 21 September reading arrived at 02:45), while **missing** readings lower totals permanently and have about ten times the effect. Daily totals should be published as provisional at midnight and final after about three hours.

9.4 Anomaly Detection

For each meter, 15-minute time slot and day type (weekday/weekend), the baseline is the median power over the 28 days. Each reading's ratio = power_kw/baseline is flagged if it is ≥ 2.0 (spike) or ≤ 0.3 (drop). The thresholds were chosen from the ratio distribution before looking at the labels: 99.99 % of ratios lie between 0.765 and 1.294.

Metric	Value
Flagged readings	474 (236 high, 238 low) on 40 meters
Injected events detected	40 / 40 (20 spikes, 20 drops)
Missed events	0
False flags	0 of 10,643,261 normal readings

Table 16: Anomaly detection compared with `anomaly_events()`

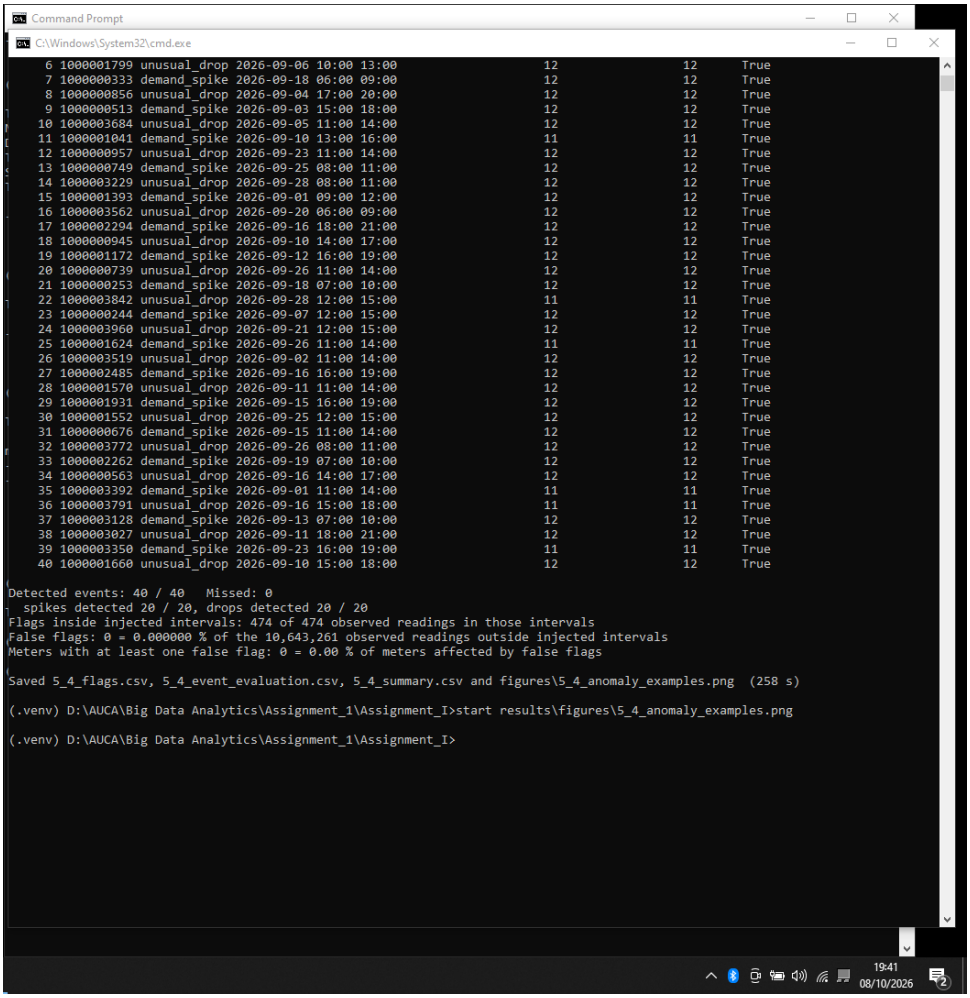


Figure 10: Example meters: actual power vs baseline during a spike and a drop

Every injected event changed power by a large factor for three hours, so all were found. A milder anomaly (for example +50 %) would be missed; this is the price of conservative thresholds.

10 Continuous Aggregates

10.1 Design

A continuous aggregate stores pre-computed group totals and refreshes them incrementally, so dashboards do not have to re-scan millions of raw rows. Three views were created with `sql/12_continuous_aggregates.sql`, then refreshed manually over the whole period by `src/caggs_refresh.py`.

View	Bucket and group	Rows	Refresh	Size
`energy_15min_region`	15 min × region	26,880	108.19 s	4.15 MB
`energy_hourly_region`	1 hour × region	6,720	52.00 s	1.23 MB
`energy_daily_meter`	1 day × meter	112,000	27.84 s	12.55 MB
Total		**145,600**	**188.03 s**	**17.93 MB**

Table 17: Continuous aggregates: rows, initial refresh time and storage

The three views use 17.93 MB, about 1.3 % of the raw table. Refresh policies were then added with a 3-day lookback, a 1-hour end offset and a 15-minute schedule.

10.2 Freshness and Delayed Arrivals

`src/freshness_test.py` inserted test readings into an older hour and the latest completed hour H, and checked the aggregate after each step.

Stage	Older hour	Hour H
1. Older hour refreshed, real-time off	1 kWh	no bucket
2. New inserts, no refresh, real-time off	1 kWh	no bucket
3. Real-time on, still no refresh	1 kWh	2 kWh
4. Both hours refreshed	1.5 kWh	2 kWh

Table 18: Aggregate results at each freshness stage (raw data: 1.5 kWh and 2 kWh)

Inserts never change stored buckets. Real-time aggregation only adds raw data *after* the newest stored bucket, so hour H appeared but the late 0.5 kWh in the older hour did not; only a refresh covering that hour corrected it. The 3-day lookback is what lets delayed readings reach the aggregates.

11 Dashboards

11.1 Raw and Aggregate Dashboards

Grafana runs in Docker and is provisioned from files. Two dashboards with the same five panels, CAT time zone and a `$meter` filter were generated by `src/dashboards.py`: the **Raw** dashboard reads `energy_readings`, the **Aggregate** dashboard reads the three continuous aggregates.

Panel	Raw dashboard reads	Aggregate dashboard reads
Hourly regional energy	`energy_readings` + `meters`	`energy_hourly_region`
Daily network energy	`energy_readings`	`energy_daily_meter`
Energy per seven-day period	`energy_readings`	`energy_daily_meter`
Daily energy, selected meter	`energy_readings`	`energy_daily_meter`
Completeness by region	`energy_readings` + `meters`	`energy_15min_region`

Table 19: Panel sources on the two dashboards

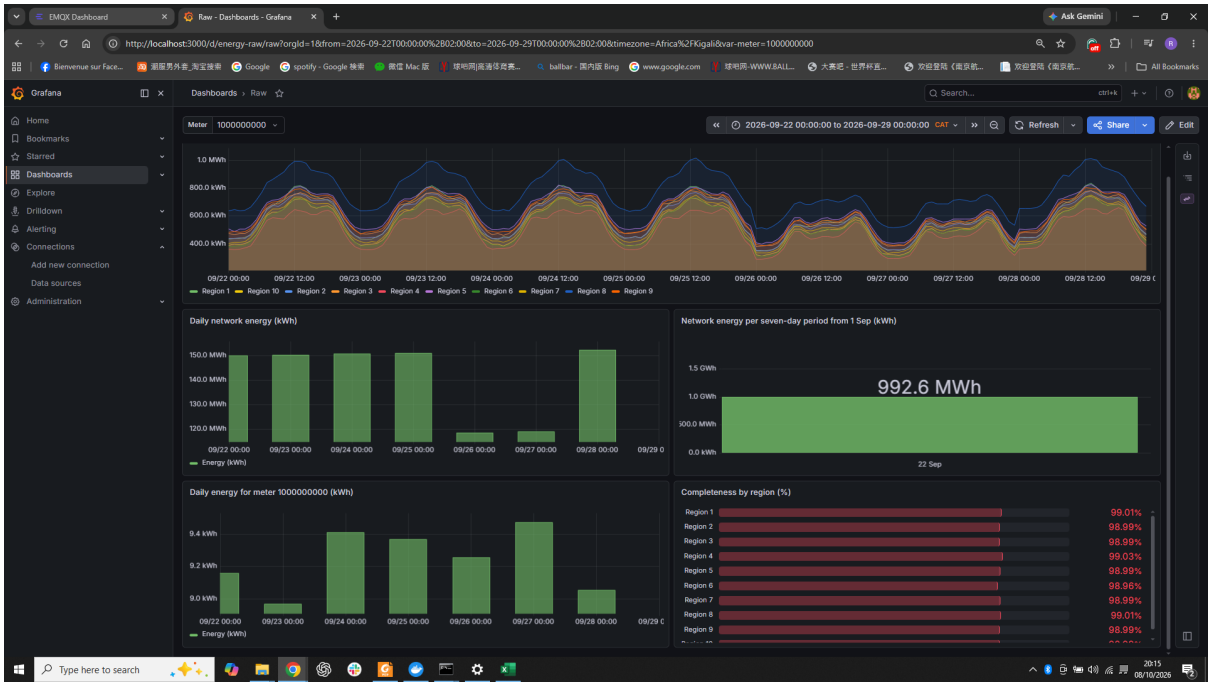


Figure 11: Raw dashboard, 22–29 September, meter 1000000000

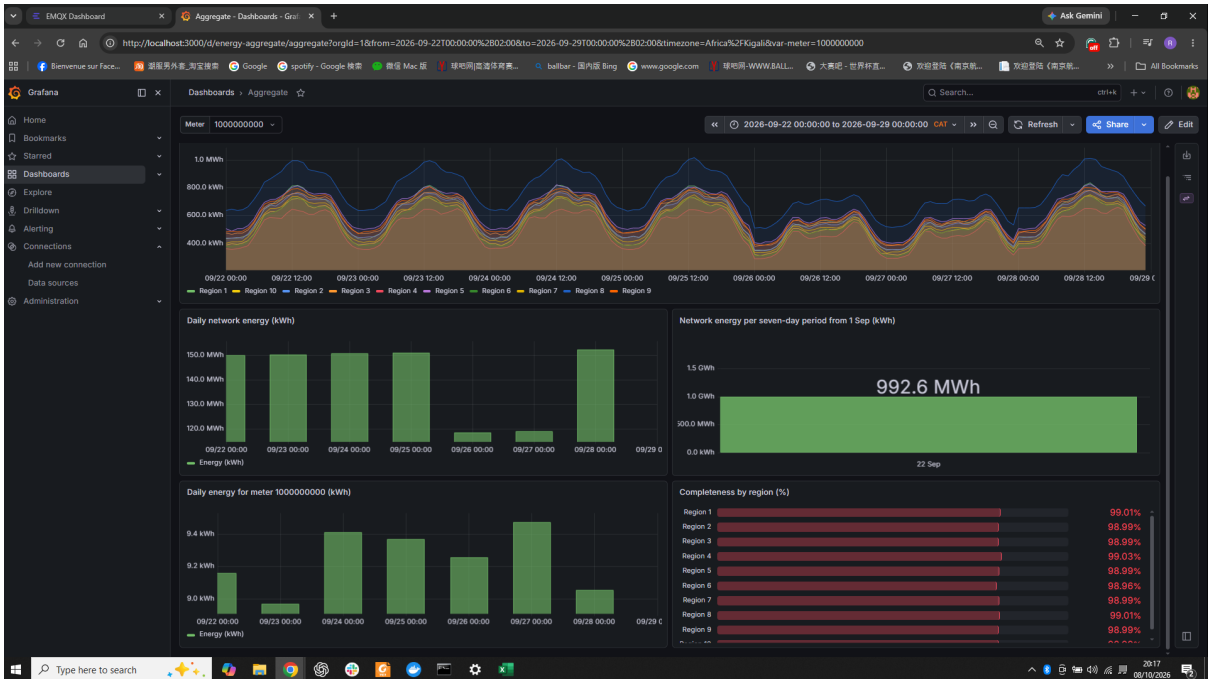


Figure 12: Aggregate dashboard, same range and meter

11.2 Verification

`src/verify_dashboards.py` sent every panel query of both dashboards through Grafana's query API and compared the answers value by value.

Panel	Values compared	Max difference	Result
Hourly regional energy	3,360	0.000000000	PASS
Daily network energy	14	0.000000005	PASS
Seven-day period	2	0.000000012	PASS
Daily energy, meter 1000000000	14	0.000000000	PASS
Completeness by region	30	0.000000000	PASS

Table 20: Raw vs Aggregate verification

Both dashboards return the same values; the tiny differences are floating-point rounding.

11.3 Performance

`src/dashboard_timing.py` drove headless Chrome with Playwright, paused all background jobs, disabled the browser cache and refreshed each dashboard once as a warm-up and then 10 measured times, alternating Raw and Aggregate.

Panel (median, ms)	Raw	Aggregate	Speed-up
Hourly regional energy	12,106	361	33.6×
Daily network energy	8,964	272	33.0×
Seven-day period	10,289	222	46.3×
Daily energy, one meter	182	249	0.7×
Completeness by region	6,400	222	28.8×
Whole dashboard	**12,891**	**493**	**26.1×**

Table 21: Dashboard refresh times over 10 measured refreshes

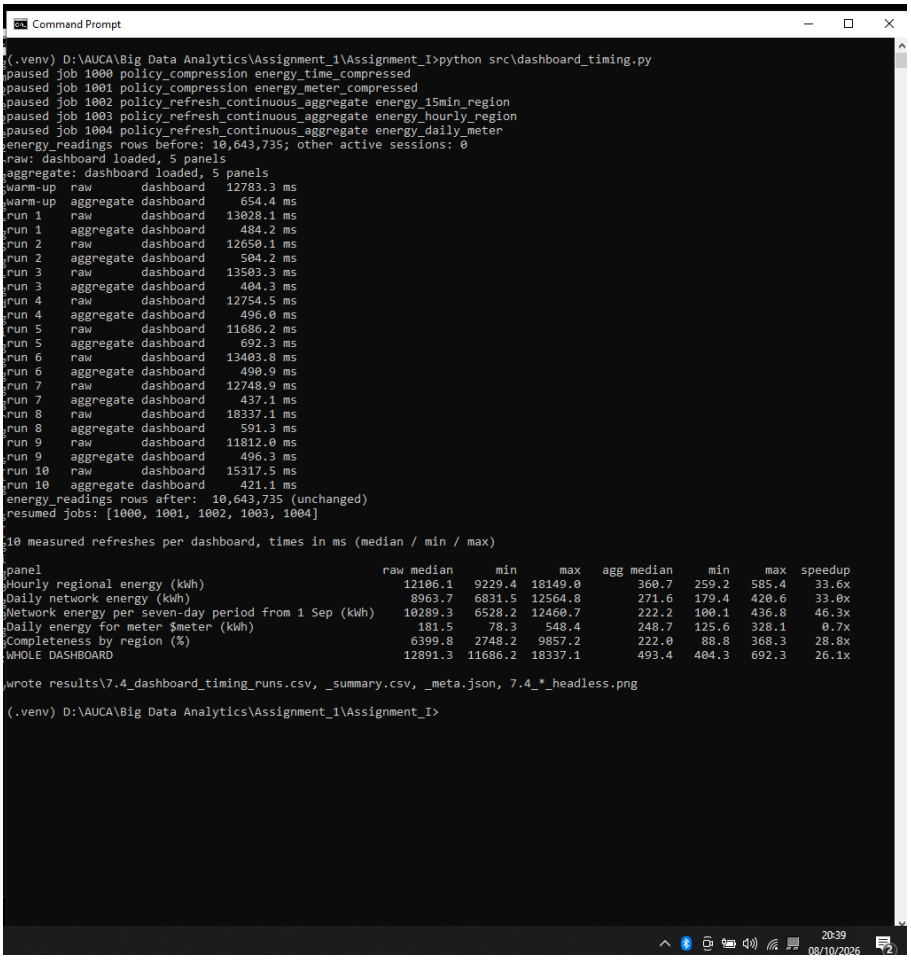


Figure 13: Dashboard timing output: per-panel and whole-dashboard statistics

The Aggregate dashboard loads in about **0.5 s instead of 12.9 s**. The single-meter panel gains nothing because on the raw table it is already a fast index lookup. The aggregates cost 17.93 MB and a one-off 188 s refresh; since each Aggregate refresh saves about 12.4 s, this cost is recovered after about 15 dashboard refreshes.

12 Day-Ahead Forecasting

12.1 Dataset and Features

The goal is to forecast each region's hourly energy for the next day, with the forecast made at 00:00 CAT. `src/forecast_8_1_dataset.py` built one row per region and hour: **6,720 rows** (10 regions × 672 hours), with totals kept as observed and nothing filled in. Every feature uses only hours that ended before the forecast origin.

Feature	Meaning
`lag_24h`, `lag_48h`	Same hour 1 and 2 days earlier
`lag_168h`	Same hour one week earlier
`prev_day_mean`	Average hour of the previous day
`prev_week_same_hour_mean`	Average of the same hour over the previous 7 days
`hour`, `dow`, `region`	Calendar and category (no lag)

Table 22: Forecast features

Split (chronological)	Rows	Usable rows
Train, 1–14 Sep	3,360	1,680
Validation, 15–21 Sep	1,680	1,680
Test, 22–28 Sep	1,680	1,680

Table 23: Chronological splits (training rows from 1–7 Sep lack a week of history)

12.2 Baseline and Model

The **baseline** forecasts each hour as the same hour of the previous day. The **model** is Ridge regression (scikit-learn) with standardised lag features and one-hot encoded region, hour and day of week. Four penalty strengths were compared on the validation week.

Config	Settings	Val. MAE (kWh)	Val. RMSE (kWh)
R1 (selected)	Ridge(α=0.1)	**4.442**	**5.645**
R2	Ridge(α=1.0)	4.454	5.660
R3	Ridge(α=10.0)	4.968	6.363
R4	Ridge(α=100.0)	8.281	11.791

Table 24: Validation comparison of Ridge configurations

R1 was saved to a file, reloaded and used for the 1,680 test predictions without any refitting.

12.3 Test Results

Method	MAE	RMSE	MAE as % of mean hour
Previous-day baseline	42.708 kWh	79.403 kWh	7.23 %
Ridge R1	**4.557 kWh**	**5.798 kWh**	**0.77 %**

Table 25: Test metrics, 22–28 September (1,680 predictions)

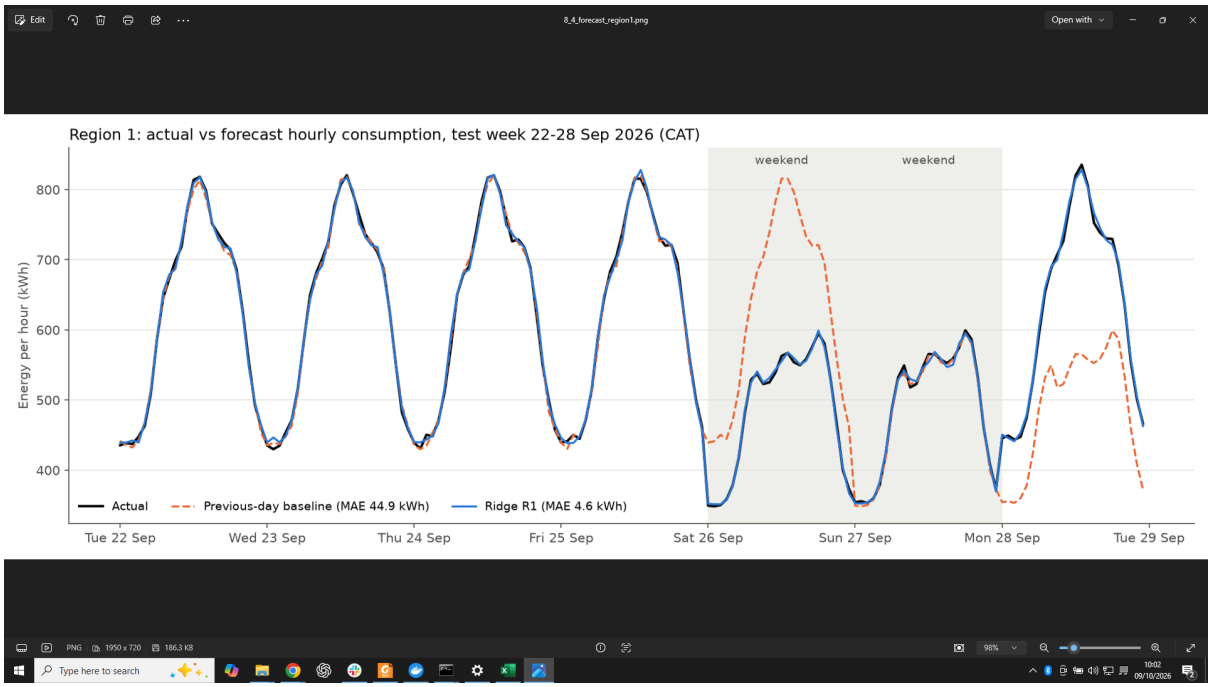


Figure 14: Region 1: actual vs baseline vs model over the seven test days

The model cuts MAE by **89.3 %** and RMSE by **92.7 %**, and wins in every region. The gain comes from the day-type switches: on Saturday and Monday the baseline misses by about 136–139 kWh per hour, while the model stays at 3.6–4.9 kWh. At each region's peak hour the mean absolute error was 6.3 kWh for the model against 159.9 kWh for the baseline. Seven of the ten regional peaks fell on Monday 28 September, exactly when the baseline fails.

13 Business Insights

This section answers the business questions from Section 2 and gives the management recommendation.

Question	Finding
Reliable ingestion?	Yes, after the fix. A slow subscriber lost 183 of 2,000 messages in the broker; with a separate database-writer thread and paced publishing, 2,000 of 2,000 arrived (confirmed again by the README rerun).
Data quality?	98.99 % complete; 0.50 % duplicates (removed); 2.00 % delayed; 1.01 % missing (kept as gaps, never zero).
Best chunk interval?	One day: never the slowest and within 2 % of the best on four of five queries.
Best compression?	Segment by `meter_id` after 48 hours: 63 % less storage, per-meter aggregation 1,304.6 → 268.4 ms, single-meter lookups still fast (4.1 ms).
When is demand highest?	Weekday middays; peak Monday 28 Sep 13:00 (7,993.7 kW). Weekends are 21.3 % lower; demand grows about 2 % per week.
Who drives demand?	Industrial customers: 10 % of meters, 57.6 % of energy. Regional differences follow the industrial share ($r = 0.97$); Region 8 is the largest.
Anomaly detection?	The median-ratio rule found 40 of 40 injected events with no false flags.
Faster dashboards?	Continuous aggregates: 26.1× faster (493 vs 12,891 ms) with identical results, for 17.93 MB of storage. Single-meter panels should stay on the raw table.
Better forecasts?	Yes. Ridge regression: MAE 4.557 kWh vs 42.708 kWh (89 % lower), peak-hour error 6.3 vs 159.9 kWh.

Table 26: Answers to the business questions

Management recommendation. Plan capacity for weekday middays and Monday restarts, and allow for continued growth. Start demand-response programmes with industrial customers and Region 8. Monitor completeness per meter daily and publish daily reports at 03:00 CAT, after late readings have arrived. Store data in one-day chunks compressed by meter after 48 hours, build dashboards on continuous aggregates, and adopt the Ridge model for day-ahead regional forecasts, run at 03:00 CAT with a peak margin of about 3 % and retrained weekly.

14 Reproducibility and Contributions

14.1 Reproducibility

The repository and the submitted ZIP archive https://github.com/Rubagumyabrave/Big_data_Analysis_Assignment_I_Smart_grid/archive/refs/heads/main.zip, created with `git archive`) contain the same files. `README.md` gives the setup and execution order: start the containers, create the database and schema, run the pilot, load and clean the data, then run the chunk, compression, analysis, aggregate, dashboard, and forecast scripts. Following README section 4 on 9 October 2026, the pilot was rerun from scratch and again reached **2,000 at every stage**.

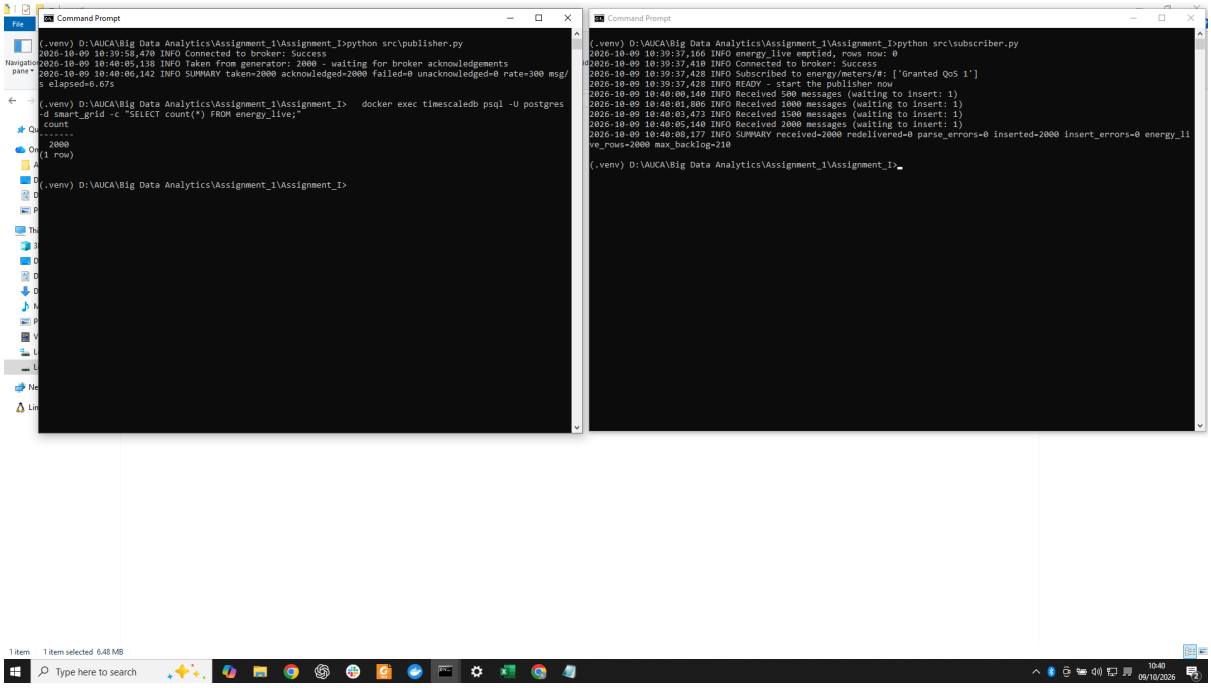


Figure 15: Pilot rerun from the README: publisher, subscriber and `energy_live` count all 2,000

14.2 Group Contributions

Member	ID	Contributions
Rwagasana Felicien	101364	Ingestion and data preparation: environment, schema, MQTT pilot and fix, bulk load, data quality, cleaning, README
Umuhoza Diane	101286	Storage and performance: chunk-interval benchmark, compression experiment, continuous aggregates, freshness test
Rubagumya Brave	101392	Analysis, dashboards and forecasting: demand, regional, quality and anomaly analysis, Grafana dashboards, forecast model
All members		Report writing and review

Table 27: Member contributions

15 Limitations

1. **Single machine:** All services share one 4-CPU Docker virtual machine, so benchmark times reflect a laptop, not a production server.
2. **Synthetic data:** The generator does not model holidays, weather or real customer behaviour changes, which would create more variation and false anomaly alarms.
3. **Short period:** 28 days of data; the forecast model was trained on only one week of usable rows and should be retrained regularly.
4. **Missing readings:** About 1 % of readings never arrive, so hourly totals and forecast targets are slightly below true demand (about 6 kWh per hour).
5. **Delayed arrivals in live use:** The forecast test used final data; at midnight the last hours of the previous day are still incomplete, so the forecast should run at about 03:00 CAT.
6. **Bulk load path:** The full dataset was loaded with `COPY` from CSV rather than through MQTT, because publishing 10.7 million messages would take hours; the MQTT path was proven with the pilot.
7. **Conservative anomaly thresholds:** The rule detects large spikes and drops; milder anomalies (for example +50 %) would be missed.

16 Conclusion

This project built and evaluated a complete smart-grid data pipeline on a single machine. The MQTT pilot showed that broker acknowledgements alone do not guarantee delivery: a slow subscriber lost 183 messages until database writing was moved to a separate thread, after which every reading arrived. The full dataset of 10,696,848 readings was loaded without loss, measured against the reference counts and cleaned of 53,113 duplicates, while missing readings were kept as gaps.

On the storage side, one-day chunks compressed by meter after 48 hours gave the best balance: 63 % less storage, two to five times faster aggregations and fast single-meter lookups. Continuous aggregates made the dashboards 26 times faster for 18 MB of extra storage, with results identical to the raw data.

The analysis showed a 21 % weekend drop, steady growth of about 2 % per week, and demand driven by industrial customers, with data 98.99 % complete and gaps spread evenly. Finally, a Ridge regression that knows the weekly pattern cut the day-ahead forecast error from 42.7 to 4.6 kWh per hour, making it useful for regional planning provided it runs after late readings have arrived.

17 References

1. TimescaleDB. (2026). *TimescaleDB Documentation: Hypertables, Compression and Continuous Aggregates*. Retrieved from https://docs.timescale.com
2. EMQX. (2026). *EMQX Documentation*. Retrieved from https://docs.emqx.com
3. Grafana Labs. (2026). *Grafana Documentation: Provisioning and PostgreSQL Data Source*. Retrieved from https://grafana.com/docs/grafana/latest/
4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
5. Temitope Ohuntade. (2026). *Advanced Big Data Analytics — Assignment I: Smart Energy Grid Analytics*. Adventist University of Central Africa (AUCA).

Execution Evidence

The key screenshots appear in the sections above. All 63 screenshots, each showing the executed command, its result, the taskbar and the clock, are stored in the `Screenshoots/` folder of the repository and listed below.

File	Shows
`env_creation`	Python virtual environment and requirements
`1.1_a_broker_running`	EMQX and TimescaleDB containers running
`1.1_b_database_extension_timezone`	Database, extension and CAT time zone
`1.1_c_environment_2`	Hardware, versions and PostgreSQL settings
`1.2_a_tables_1`, `1.2_a_tables_2`	Table definitions and hypertable chunk interval
`1.2_c/d_meters_count`	4,000 meters loaded
`1.3_run1_emqx_dropped`	Run 1: 183 messages dropped by the broker
`1.3_run2_publisher_subscriber`	Run 2: 2,000 published and received
`1.3_run2_emqx_metrics`	Run 2: no new drops
`1.3_db_count_sample`	2,000 rows in `energy_live`
`2.1_a` to `2.1_d`	Expected counts, generation, load, database checks, chunk ranges
`2.2_a_dq_summary`, `2.2_b_delays_top_missing`	Data-quality counts, delays, worst meters

File	Shows
2.3_a to 2.3_c	Duplicate investigation, cleaning result, repeat run
3.1_a to 3.1_c	Building and checking the three chunk configurations
3.2_0_jobs_before, 3.2_a_benchmark_results	Background jobs and benchmark medians
3.2_b_plan_3h, 3.2_d_plan_week	Query plans on the 3-hour and 7-day tables
4.1_a to 4.1_e	Compression set-up, settings, policies and checks
4.2_a_storage, 4.2_b_storage	Storage and compression ratios
4.3_a_benchmark_results	Compressed vs uncompressed query medians
4.3_b_plan_time_compressed_top/bottom	Q4 plan on the time-ordered table
4.3_c_plan_meter_compressed	Q4 plan on the meter-segmented table
4.3_d_policies_resumed	Compression policies resumed
5.1_a_analysis_output	Daily, weekly and peak-interval results
5.2_a_group_counts_comparison	Customer type and region comparison
5.3_a_completeness_daily_totals	Completeness and on-time vs final totals
5.4_a_detection, 5.4_b_evaluation	Anomaly thresholds, flags and evaluation
5.4_c_examples_chart	Spike and drop examples
6.1_a to 6.1_d	Aggregate creation, refresh, definitions, policies
6.2_a_definitions, 6.2_b_stages	Freshness test set-up and stages
7.1_raw_dashboard,	The two dashboards
7.2_a_aggregate_dashboard	
7.2_b_panel_queries, 7.2_c_raw_panel_query	Aggregate and raw panel queries
7.3_dashboard_verification	All five panels PASS
7.4_a_timing_output, 7.4_b_jobs_resumed	Dashboard timing and jobs resumed
8.1_dataset_features_splits	Forecast dataset, features and splits
8.2_baseline	Baseline predictions and metrics
8.3_model_validation_test	Ridge validation and test predictions
8.4_a_metrics_peak_hours,	Final comparison and forecast plot
8.4_c_forecast_plot	
9.2_pilot_rerun	Pilot rerun from the README

Table 28: List of execution screenshots